CircuitNet: An Open-Source Dataset for Machine Learning in VLSI CAD Applications With Improved Domain-Specific Evaluation Metric and Learning Strategies

Zhuomin Chai, Yuxiang Zhao, Wei Liu, Yibo Lin, *Member, IEEE*, Runsheng Wang, *Member, IEEE*, and Ru Huang, *Fellow, IEEE*

***Abstract*—The design automation community has been actively exploring machine learning (ML) for very-large-scale-integrated (VLSI) computer-aided design (CAD). Many studies have explored learning-based techniques for cross-stage prediction tasks in the design flow. Although building ML models usually requires a large amount of data, most studies can only generate small internal datasets for validation due to the lack of large public datasets. Such a situation challenges the research in this field and raises potential issues like difficulty in benchmarking and reproducing results, limited research scope on small internal datasets, and high bar for new researchers. Therefore, in this article, we present an open-source dataset called "CircuitNet" for ML tasks in VLSI CAD. The dataset consists of more than 10K samples extracted from versatile runs of commercial design tools based on six open-source RISC-V designs which support typical cross-stage prediction tasks, such as routability and IR drop prediction, with extensive benchmarking on recent models. With the dataset prepared, we identify two practical challenges, data imbalance and model transferability, for ML application in CAD. To overcome data imbalance, we propose a loss function, *biased loss*, to give more weight to the minority, leading to 2% congestion reduction in routability-driven placement. We test the model transferability from RISC-V designs to ISPD 2015 contest designs in congestion prediction with several transfer learning methods and further proposed a knowledge distillation-based transfer learning framework with up to 20% accuracy improvement. We believe this dataset can open up new opportunities for ML in CAD research and beyond.**

***Index Terms*—Benchmark testing, integrated circuit modeling, physical design, predictive models, training.**

Manuscript received 30 December 2022; revised 16 April 2023; accepted 28 May 2023. Date of publication 20 June 2023; date of current version 22 November 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2021ZD0114702; in part by the National Science Foundation of China under Grant 62125401 and Grant 62034007; in part by the 111 Project under Grant B18001; in part by the Special Fund of Hubei Luojia Laboratory under Grant 220100025; and in part by the Fundamental Research Funds for the Central Universities under Grant 413000137. This article was recommended by Associate Editor I. H.-R. Jiang. *(Corresponding authors: Wei Liu; Yibo Lin.)*

Zhuomin Chai is with the School of Physics and Technology and the School of Microelectronics, Wuhan University, Wuhan 430072, China, also with the School of Integrated Circuits, Peking University, Beijing 100871, China, and also with Hubei Luojia Laboratory, Wuhan 430072, China.

Yuxiang Zhao is with the School of Integrated Circuits, Peking University, Beijing 100871, China.

Wei Liu is with the School of Physics and Technology and the School of Microelectronics, Wuhan University, Wuhan 430072, China, and also with Hubei Luojia Laboratory, Wuhan 430072, China (e-mail: wliu@whu.edu.cn).

Yibo Lin, Runsheng Wang, and Ru Huang are with the School of Integrated Circuits, Peking University, Beijing 100871, China, also with the Institute of Electronic Design Automation, Peking University, Wuxi 214028, China, and also with the Beijing Advanced Innovation Center for Integrated Circuits, Beijing 100871, China (e-mail: yibolin@pku.edu.cn).

Digital Object Identifier 10.1109/TCAD.2023.3287970

I. Introduction

MODERN very-large-scale-integrated (VLSI) circuit design has a long design flow. To avoid iterative information feed-forward and feed-backward between design stages for convergence, which is extremely expensive due to the time-consuming feedback loops, the common practice is to incorporate a lookahead estimation, such as the global router for routability evaluation during global placement [1], [2], [3]. However, as the advance in semiconductor technology along with the increasing design scale and complexity [4], the design flow of modern VLSI circuits is becoming much harder to predict, especially, for the design objectives, such as timing, wirelength, routability, power, IR drop, etc. Thus, the common estimation methods have become neither accurate nor fast.

Machine learning (ML) brings new opportunities for VLSI computer-aided design (CAD) applications [5], [6], [7], where the traditional lookahead estimation with a heuristic algorithm can be potentially replaced with ML models which provide fast and accurate cross-stage prediction, as shown in Fig. 1. Such an ML-assisted design flow enables early-stage prediction of design quality and is promising to significantly speed-up design closure and improve the eventual quality of results (QoR).

Most studies try to investigate fast and accurate cross-stage prediction tasks with ML in the back-end design, as placement and routing iterations are extremely time-consuming and an accurate prediction of design objectives can effectively reduce required iterations. To be specific, early-stage prediction for routability and IR drop has been extensively explored.

Routability prediction can be roughly categorized into two tasks: prediction of routing congestion and design rule check (DRC) violations. Predicting routing congestion aims at guiding placement optimization for routability. A typical approach is to formulate the problem into a computer vision task and leverage generative models to learn the correlation [8], [9],

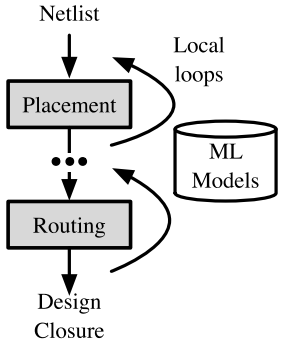


Fig. 1. Example of a simplified back-end design flow with ML-assisted.

[10], [11], [12], [13], [14], [15], [16], [17], [18], as the layout information can be naturally represented with image-like data. Predicting DRC violations aims to resolve detailed routability issues. The literature has explored support vector machine, random forests, and deep learning models to build the correlation between global routing solutions and DRC violations after detailed routing [10], [19], [20], [21], [22], [23], [24], [25]. Besides the prediction of routing congestion and DRC violations, studies have also been carried out to predict routed wirelength during even earlier stages like logic synthesis [26]. On the other hand, IR drop prediction is to evaluate the supply voltage drop on the power grids. IR drop consists of static and dynamic IR drops. Static IR drop is related to the distribution of power grids, while dynamic IR drop is correlated to the distribution and power demand of standard cells. As IR drop can increase cell delay and cause timing degradation, previous works have proposed efficient ML models for IR drop prediction [27], [28], [29], [30], [31], [32].

Despite the active research in this field, ML for CAD is also encountering challenges.

1) Above all, from the perspective of dataset, unlike in computer vision research where validation on large-scale public datasets, such as ImageNet [33], CIFAR [34], etc., is a common practice, there is almost no public dataset dedicated to ML for CAD applications due to the license restriction and domain-specific expertise for data generation. Meanwhile, existing datasets from CAD contests are often incomplete and not designed for ML applications [35], [36], [37]. Although building an ML model generally requires a large amount of data, most studies can only generate small internal datasets with at most a few thousand samples for validation. The lack of public datasets raises challenges like difficulty in benchmarking and reproducing previous work, limited research scope from limited data access, and a high bar for new researchers, which slows down further advancements in this field.
2) Besides, in terms of model training, the data imbalance problem is prevailing in the CAD field. For example, only a small portion of a design is congested or contains many DRC violations. A highly imbalanced dataset will lead the model to learn from the majority and ignore the minority that we really care about but do not contribute much to the training loss, as model training is formulated as an overall loss across all training samples.
3) Finally, effective knowledge transfer between different designs and technology nodes is rather important in ML for CAD, because data generation is expensive and may involve multiple design stages. But transfer learning techniques other than the basic pretraining and fine-tuning strategy are rarely explored, and most work still uses the small internal dataset to verify the transferability.

In this work, we present an open-source dataset, *CircuitNet* [38], for ML tasks in CAD applications, especially, for cross-stage prediction tasks. We generate over 10K layout samples with 54 synthesized circuit netlists from 6 open-source RISC-V designs, using commercial design tools with well-designed parameter settings to introduce variations in a commercial 28-nm technology node. We extract a variety of features from the runs of the commercial design flow at different design stages, enabling multiple cross-stage prediction tasks, including but not limited to congestion prediction, DRC violation prediction, and IR drop prediction. Based on the CircuitNet dataset, we conduct extensive benchmarking on recent studies for cross-stage prediction tasks. Then, we further propose two techniques to address the data imbalance problem and the model transferability problem. First, we design a new loss function to enable biased learning in routability prediction and IR drop prediction. The proposed loss function can improve performance in these tasks without modifying the network structure. Second, we explore the transferability of models between different sets of designs (i.e., RISC-V designs and ISPD 2015 contest designs [36], widely used for routability optimization [2], [3], [16], [39]) with transfer learning techniques.

We highlight our contributions.

1) We present an open-source dataset, CircuitNet,[1] dedicated to ML applications for CAD problems, in attempting to address cross-stage prediction tasks like routability and IR drop in the back-end design flow [38]. The dataset consists of over 10K samples and 54 synthesized circuit netlists, providing holistic support for cross-stage prediction tasks. We provide both extracted features for immediate application and sanitized LEF/DEF files for user-defined applications.
2) We propose an improved domain-specific metric, peak NRMSE, to emphasize the congested regions for routability optimization. In addition, we design a biased loss to deal with the data imbalance problem in congestion prediction. Experimental results on DRC violation prediction and IR drop prediction demonstrate that the proposed loss can lead to 5% TPR improvement at about 5% FPR. Integrating the congestion model into placement optimization can contribute to 2% reduction in routing congestion, indicating the consistency between the peak NRMSE metric and practical scenarios for congestion optimization.
3) We further propose a transfer learning framework, utilizing a teacher–student network to improve transferability between RISC-V designs and ISPD 2015 contest designs [36]. Experimental results demonstrate the

[1] https://circuitnet.github.io/

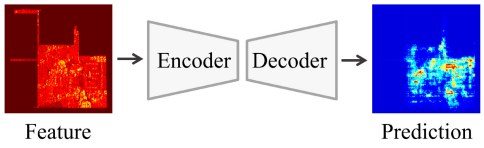


Fig. 2. Encoder–decoder architecture for prediction tasks in the back-end design.

proposed method outperforms the widely adopted transfer learning techniques by up to 20% in NRMSE, 10% in SSIM, and 5% in peak NRMSE.

The remainder of this article is organized as follows. Section II introduces the background of ML in VLSI CAD applications. Section III shows detailed steps of constructing datasets. Section IV presents the basic flow of routability and IR drop prediction, the algorithm of the proposed evaluation metric and loss function, and a transfer learning framework. Section V validates the dataset and the proposed techniques with experimental results. Finally, Section VI concludes this article.

II. Preliminary

In this section, we review the background of ML in VLSI CAD applications and formulate the problems.

A. Back-End Design Flow

VLSI design flow can be divided into front-end design and back-end design. A typical front-end design flow consists of functional design and logic synthesis, converting a circuit design into a gate-level circuit netlist. A typical back-end design flow consists of floorplanning, placement, clock-tree synthesis, routing, etc., mapping a synthesized netlist into a physical layout for manufacturing. In advanced technology nodes, a complete back-end design flow is extremely time-consuming due to iterative optimization for design objectives. Therefore, introducing ML models for fast and accurate cross-stage prediction is promising to speed up design closure and improve QoR.

As back-end design works on physical layouts, the design information can be naturally represented with image-like data by dividing a layout into tiles and regarding each tile as a pixel. To this end, a prediction task can be formulated into an image-to-image translation task, enabling the application of generative models, such as fully convolutional networks (FCNs) and conditional generative adversarial networks (cGANs). All these models share an encoder–decoder architecture, as shown in Fig. 2.

On the other hand, the netlist possesses connectivity information, i.e., the nets between cells, which cannot be represented in the image form, and this information is also important for applications. In this work, we mainly focus on image-like data and models with encoder–decoder architecture, but CircuitNet also provides support for graph-based applications.

B. ML for Routability Prediction

Routability refers to the prospective quality of routing based on the current design solution. As the design stages

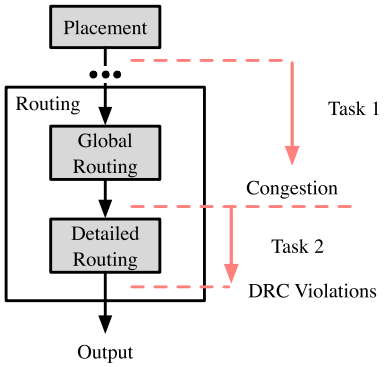


Fig. 3. Two routability prediction tasks: congestion prediction and DRC violation prediction.

are time consuming and the design flow needs to be iterated until convergence is achieved, early-stage routability prediction is an important method to guide and accelerate the optimization. Routability prediction tasks include congestion prediction and DRC violations prediction. At the placement stage, routing congestion prediction is critical to guide placement optimization. It reports the overflow amount and locations where the routing demands exceed the available routing resources. DRC violations prediction is proposed to handle the miscorrelation between congestion and DRC violations [19] to resolve detailed routability issues. The literature has explored both tasks with various ML techniques. The basic flow of routability prediction is presented in Fig. 3.

For congestion prediction, Liu et al. [16] and Chen et al. [15] integrated an FCN-based model into the placer to achieve fast routability-driven placement. Yu and Zhang [8] encoded connection as flying lines into an image as the interconnect feature to predict congestion with conditional generate adversarial networks (cGANs). This work tries to incorporate graph-based interconnection information with spatial information on routing demands. Alawieh et al. [12] improved the feature representation and the scalability of the cGAN models for large-scale circuits.

For DRC violation prediction, Chan et al. [19] took each global routing tile as input to predict the existence of DRC violations with regression and support vector machine. Xie et al. [20] discovered the impact of macro locations on routability and propose RouteNet, an FCN-based model that encodes the entire layout features as an image-like tensor, to predict the map of DRC violations. Studies mentioned above need congestion information from global routing as input to make predictions. Liang et al. [10] further proposed JNet which can directly predict DRC violations from placement solutions without global routing congestion, utilizing a high-resolution pin configuration map as one of the features and a modified U-Net architecture.

All the aforementioned studies require common features, such as pin density, cell density, and rectangular uniform wire density (RUDY) [40] as inputs for both congestion and DRC violation prediction tasks. For DRC violation prediction, we further need global routing congestion or pin configuration map as additional inputs.

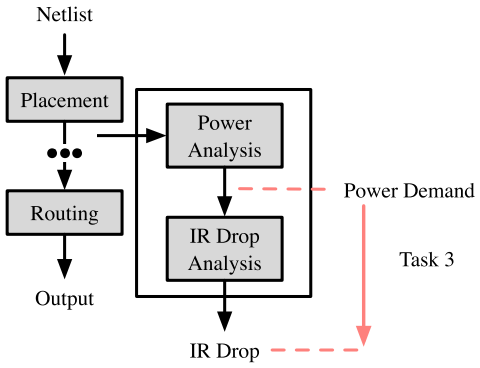


Fig. 4. Typical tool flow of IR drop analysis and IR drop prediction task.

C. ML for IR Drop Prediction

IR drop is defined as the deviation of voltage from reference (VDD and VSS) and it has to be restricted to avoid degradation in timing and functionality. IR drop can be categorized into two types: static and dynamic IR drops. Static IR drop is time-invariant, as it is derived from voltage drop caused by the resistance on power grids. Dynamic IR drop is time-variant, as it is caused by switching activities of standard cells and local current fluctuations. Accurate simulation of IR drop requires solving large-scale ordinary differential equations on power grids, which is extremely time consuming. Moreover, as IR drop has correlated with the distribution of standard cells and repeated invocation of IR drop simulation is unaffordable, fast and accurate IR drop prediction with ML models is desired to reduce the turnaround time.

Fig. 4 shows the flow of recent efforts on IR drop prediction with the ML model. Xie et al. [29] proposed PowerNet, based on a max-CNN structure, to capture the max power demand among all timing windows of arrived signals. Huang et al. [30] introduced careful feature engineering strategies to consider cumulative current and form an accurate relationship between individual power and IR drop. Chhabria et al. [32] further proposed MAVIREC, which formulates an image-to-image translation problem and explores 3-D convolution layers to encode timing information with a U-Net architecture.

D. Data Imbalance

Data imbalance refers to the problem in predicting the minority class caused by the unequal distribution of classes in the dataset [41], [42]. The unequal distribution of classes will lead the model to relatively ignore the minority that contributes little to the loss, as the ultimate goal of training ML model is to minimize the loss. There are mainly two strategies for solving this problem, resampling and reweighting, and they have been widely explored in ML for CAD for applications like lithography hotspot detection, yield prediction, DRC violation prediction, wafer defect classification, and net delay prediction [43], [44], [45], [46], [47], [48], [49]. Resampling methods aim at balance class distribution in the dataset, including oversampling [43], [44] and undersampling [46]. Reweighting involves manipulating the loss function to make the model more sensitive to the minority [45], [47], [48], [49].

E. Transfer Learning

Transfer learning aims at sharing the knowledge learned from a source domain to the task of a target domain [50], [51]. A typical transfer learning method is the pretraining and fine-tuning paradigm, e.g., using a pretrained backbone from ImageNet and then fine-tuning some weights of neural networks to downstream tasks [52]. As it is expensive to generate data for ML for CAD applications, transfer learning is promising to improve the generalization on small datasets.

Besides fine-tuning, regularization-based methods [53], [54] are also introduced to transfer learning. Denote the weights in the fine-tuned model as ω and the ones in the pretrained model as ω^*, then the distance of current weight ω and the starting point (SP) can be inductively represented as the L2-SP regularization

$$R(\omega) = \frac{\alpha}{2}\|\omega - \omega^*\|_2^2. \tag{1}$$

However, in this way, the weights are evenly regularized and may lead to a severe loss in accuracy. [53] proposed a method named elastic weight consolidation (EWC), which use the Fisher information matrix to evaluate the importance of each weight and gives different penalty in updating different weights to enlarge the exploration space in fine-tuning. The corresponding regularization is called L2-SP-Fisher, which can be represented as

$$R(\omega) = \sum_i \frac{\alpha}{2} F_i\left(\omega_i - \omega_i^*\right)^2 \tag{2}$$

where F_i is the Fisher information matrix of ω_i^*.

In ML for CAD, many works take transfer learning into consideration as the generalization between different designs and technology nodes is always been a problem. Overall, most of them only adopt basic pretraining and fine-tuning without further modification. For example, RouteNet [20] leverages a pretrained ResNet18 on ImageNet and fine-tunes it for predicting DRC violations after detailed routing. Reference [55] explores how to fix and fine-tune layers for lithography modeling between different technology nodes. Reference [56] investigates the strategy to fix layers for lithography hotspot detection. In [57], adaptive sampling is used to assist transfer learning in modeling and optimizing analog and mixed signal circuits. Reference [58] incorporates the transfer Gaussian process model to transfer knowledge from existing tool parameter combinations to unseen ones for tuning tool parameters. Reference [59] utilizes a teacher–student network for transferring reinforcement learning policies for net ordering in detailed routing.

F. Problem Formulation

CircuitNet aims at generating a holistic dataset for ML in CAD applications, especially, for cross-stage prediction tasks in the back-end design flow; it should be large-scale and diverse, at the same time reflecting the data distributions of real design cases.

We further define three typical cross-stage prediction tasks. For a layout divided into $w \times h$ tiles and f denotes the number of channels of features in input, the tasks can be stated as follows.

TABLE I
STATISTICS OF DESIGNS AND VARIATIONS INTRODUCED DURING LOGIC SYNTHESIS AND BACK-END DESIGN

Design	Netlist Statistics			Synthesis Variations		Back-end Design Variations			
	#Cells	#Nets	Cell Area (μm^2)	#Macros	Frequency (MHz)	Utilizations (%)	#Macro Placement	#Power Mesh Setting	Filler Insertion
RISCY-a	45,717	47,759	65,739	3/4/5	50/200/500	70/75/80/85/90	3	8	After Placement /After Routing
RISCY-FPU-a	65,793	68,351	75,985						
zero-riscy-a	34,299	33,970	58,631						
RISCY-b	31,311	33,970	69,779	13/14/15					
RISCY-FPU-b	51,126	54,327	80,030						
zero-riscy-b	20,946	22,692	62,648						

Problem 1: Given a placed design, build a model to predict congestion based on overflow levels after global routing with maximum accuracy

$$g_{\text{CG}} : X \in \mathbb{R}^{w\times h\times f} \rightarrow Y \in \mathbb{R}^{w\times h}. \tag{3}$$

Problem 2: Given a global routed design, build a model to predict DRC violations after detailed routing. This problem is formulated as a DRC hotspot detection task, with a threshold to categorize DRC hotspot regions and nonhotspot regions

$$g_{\text{DRC}} : X \in \mathbb{R}^{w\times h\times f} \rightarrow V_i \in \{0, 1\}^{w\times h}. \tag{4}$$

Problem 3: Given a placed design and power analysis report, build a model to predict IR drop. Similar to Problem 2, this problem is also formulated as an IR drop hotspot detection task

$$g_{\text{IR}} : X \in \mathbb{R}^{w\times h\times f} \rightarrow V_i \in \{0, 1\}^{w\times h}. \tag{5}$$

For Problem 1, we adopt the same accuracy metrics as those in previous work [12], [16], i.e., NRMSE and SSIM, which are typical metrics to evaluate image similarity. To capture the most congested regions for routability optimization, we further propose a new accuracy metric named peak NRMSE in the next section.

Definition 1 [Normalized Root-Mean-Square Error (NRMSE)]: Mean-square error (MSE) is the frequently used estimation of the per-pixel difference between label and prediction. And NRMSE further takes the rooted and normalized result for comparison between samples with different dimensions and distributions. The NRMSE is the smaller the better, as a lower NRMSE indicates less residual. The normalization method we use here is min–max normalization

$$\text{NRMSE} = \frac{1}{y_{\max} - y_{\min}} \sqrt{\frac{\sum_{i=1}^{h}\sum_{j=1}^{w}\left(y_{i,j} - \widehat{y}_{i,j}\right)^2}{h \times w}}. \tag{6}$$

Definition 2 [Structural Similarity Index Measure (SSIM)] [60]: SSIM indicates the structural similarity based on a $k \times k$ sliding window between two images $Y = g(X'; W^*)$ and Y'. The SSIM is the larger the better, as a larger SSIM indicates a higher structural similarity

$$\text{SSIM} = \frac{(2\mu_Y\mu_{Y'} + C_1)(2\delta_{YY'} + C_2)}{\left(\mu_Y^2 + \mu_{Y'}^2 + C_1\right)\left(\delta_x^2 + \delta_y^2 + C_2\right)} \tag{7}$$

where μ is the mean and δ is the standard deviation.

For Problems 2 and 3, we adopt the receiver operating characteristic (ROC) curve to estimate the result.

Definition 3 (Confusion Matrix): A confusion matrix shows whether the model is mislabeling positive as negative or vice versa in classification. True positive (TP), false positive (FP), true negative (TN), and false negative (FN) can be derived from the confusion matrix. For ML models that originally output a prediction value between [0,1], a decision threshold ϵ is applied to convert the model into a binary classifier.

Definition 4 (ROC Curve): TP rate (TPR) and FP rate (FPR) denote the percent of correctly/wrongly classified positive examples among all positive examples. TPR and FPR can be calculated as TPR = TP/(TP+FN) and FPR = FP/(FP+TN). By plotting TPR against FPR at various decision thresholds ϵ, a ROC curve can be obtained to illustrate the performance of the model. A larger area under curve (AUC) indicates better performance.

III. CIRCUITNET DATASET CONSTRUCTION

Datasets provide data for training and validating ML models and they serve as the foundation upon which ML algorithms are developed. In practice, some common properties of datasets are under consideration, including *scale*, *diversity*, and *accuracy* [33].

The scale of datasets should be at least on the order of magnitude of thousands. With a larger scale dataset, models with higher expressiveness can be adopted to reach better performance. Diversity typically refers to the number of label categories. However, in cross-stage prediction tasks, we care about the diversity in distribution rather than categories. Accuracy guarantees the basic function of a dataset, and it is ensured through data cleaning to remove low-quality or wrongly labeled data.

In this section, we explain how we construct CircuitNet [38], shedding light on how these three properties are considered. We complete the basic flow of synthesis and back-end design according to industrial methodology with commercial tools.

A. Collecting Data

The first stage involves generating layouts and dumping valuable reports from tools in each part of the flow.

1) Design Modification and Synthesis: The open-source platform named Pulpino [61] is adopted as the basic RTL design in CircuitNet. It is selected among several candidate open-source platforms for its simplicity and ease of use, while being a complete RISC-V SoC system, providing sufficient space to introduce variation in data.

Table I shows the statistics of the mapped netlist obtained through logic synthesis. Three original register transfer level (RTL) designs from Pulpino are used for logic synthesis with Synopsys Design Compiler. `RISCY` and `zero-riscy` refer

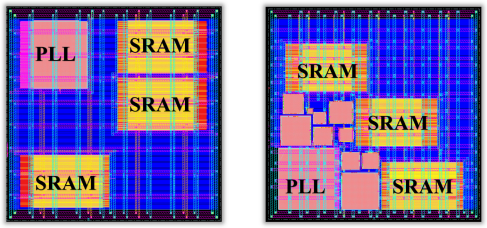


Fig. 5. Sample layouts of (a) RISCY-a-2 and (b) RISCY-b-2. "RISCY" is the design name; "-a" and "-b" mean whether the layout has *modification 2*; "-2" refers to one macro is added with *modification 1*.

to different RISC-V cores, and `FPU` refers to the use of a floating-point unit (FPU) extension.

By default, Pulpino has three macros which are not enough to represent the effect of macros in modern design. Hence, the three used RTL designs are modified in two ways to introduce extra macros.

1) *Modification 1:* One or two additional SRAMs are added into the mapped netlist directly. The original setup is denoted as "−1" and adding one macro is denoted as "−2," adding two macros is denoted as "−3."
2) *Modification 2:* Ten extra macros are introduced through hierarchical design flow. Designs without and with this modification are denoted as "a" and "b," respectively, and a more detailed difference between them can be seen in Fig. 5.

With different combinations of the modifications above, and three different timing constraints, which run the core at 50, 200, and 500 MHz, respectively, 54 gate-level netlists are synthesized from six RTL designs in the 28-nm technology node for the succeeding back-end design, as shown in Table I. Different timing constraints and the addition of macro do not influence the statistics like the number of cells and nets in the table.

2) Back-End Design: Back-end design is done with Cadence Innovus v20.10 and Voltus v20.10. To enlarge and diversify CircuitNet, each obtained gate-level netlist is placed and routed many times with different settings, which added up to a total of 240 combinations. The setting variations are listed as follows.

1) Five core utilization settings, i.e., {70%, 75%, 80%, 85%, 90%}.
2) Three macro placement from automatic floorplan performed by Innovus.
3) Eight combinations of parameters to generate power mesh.
4) Two filler insertion strategies, i.e., inserting before routing or after routing.

These settings contribute to variations in utilization, routing resources, macro locations, etc., reflecting diverse situations in the back-end design flow. For each design in Table I, there is a total of 2160 combinations of variations, summing up to 12 960 runs of back-end design in Innovus. Finally, we have 10 242 layouts for feature extraction, since some combinations lead to failure in placement and routing.

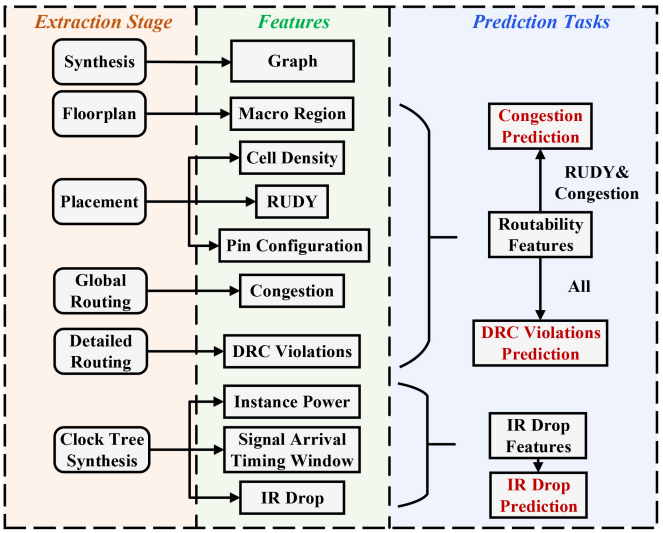


Fig. 6. Available features, their extraction stages and applications.

B. Feature Extraction

After detailed routing, all the information in CircuitNet has been generated. To convert raw data into 2-D tile-based feature maps for model training, further transaction is necessary. However, even though feature engineering is necessary to correctly capture the connection between input and output in prediction, we cannot include all existing features as every study may have a different feature design. Therefore, we extract features, as shown in Fig. 6, that are consistent with the ones used in studies in Section V and turn them into 2-D feature maps. Moreover, part of raw data and graph features are also included to enable user-specific feature extraction, and graph-based or multimodal studies which is a trend recently [17], [18], [25], [26].

1) *Raw Data:*
   a) *Netlist:* The 54 netlist synthesized in Section III-A to provide interconnectivity information for graph-based research.
   b) *LEF&DEF:* The LEF file used in physical design, and the DEF files dumped after placement to enable user-specific feature extraction.
2) *Routability Features:*
   a) *Macro Region:* The regions covered by macros which are fixed during placement and take up the routing resources, used for estimating routing resources available in each tile.
   b) *Cell Density:* The cell number counted in each tile.
   c) *RUDY:* A routing demand estimation for each net over spatial dimension. It is widely used for its high efficiency and accuracy. A variation named pin RUDY from [16] and [20] is also included as the pin density estimation.
   d) *Pin Configuration:* A high-resolution representation of pin and routing blockage shapes that conveys pin accessibility in routing.
   e) *Congestion:* The overflow of routing demand in each tile.
   f) *DRC Violations:* The number of DRC violations in each tile.

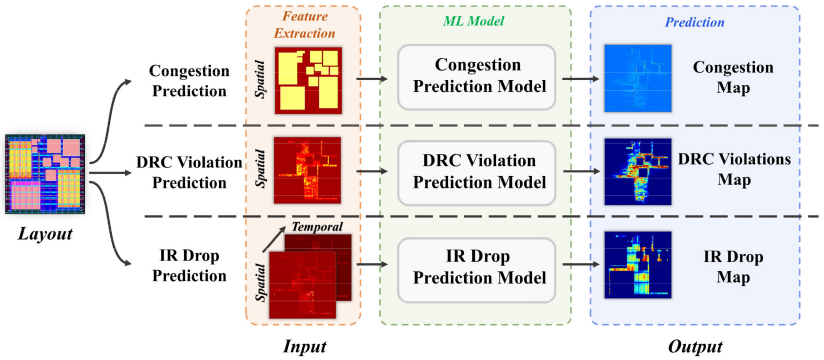


Fig. 7. Overview of the prediction tasks.

3) *IR Drop Features:*
   a) *Instance Power:* The instance-level internal, switching, and leakage power along with the toggles rate from a vectorless dynamic power analysis.
   b) *Signal Arrival Timing Window:* The possible switching time domain of the instance in a clock period from a static timing analysis for each pin.
   c) *IR Drop:* The IR drop value on each node from a vectorless dynamic power rail analysis.
4) *Graph Features:*
   a) *Graph:* The graph extracted from the netlist.
   b) *Instance Placement:* The location of all instances, i.e., macros and standard cells, extracted from the DEF files.

The tile size for all feature maps is 1.5 μm $\times$ 1.5 μm, which is the size of the global routing cell (Gcell) automatically generated by Innovus with 15 times metal 3 track length (0.1). Note that all the standard cell names in the netlist and LEF&DEF are encrypted.

To sum up, the three aforementioned properties of the ML dataset are ensured in CircuitNet.

1) *Scale:* We have over 10K samples of features collected from versatile runs of commercial tools.
2) *Diversity:* We introduce diversity through different settings, including both loose and tight constraints, in logic synthesis and back-end design flow to cover diverse situations. As a result, there is at least an order of magnitude difference in mean and standard deviation between the highest and lowest values in different samples.
3) *Accuracy:* Our data is first cleaned during feature extraction and further verified in the following benchmarking.

IV. Routability and IR Drop Prediction

In this section, we present the basic flow of routability and IR Drop prediction [38]. As shown in Fig. 7, features are extracted from the layout following the flow in Section III, then preprocessed and fed into the ML model to get the prediction.

A. Prediction Model

To validate the dataset for benchmarking, we inherit the models from selected previous studies [10], [12], [16], [20], [29], [32], and adopt the same feature extraction strategies. All the selected models are generative models with an encoder–decoder structure, including an FCN, U-Net, and cGAN.

1) FCN and U-Net: FCN [62] is a kind of convolutional neural network (CNN) first proposed for semantic segmentation. Both CNN and FCN apply downsampling layers to extract features of the input. The main difference lies in their output layers where CNN has fully connected layers to produce a scalar, and FCN has upsampling layers to reconstruct the spatial information. This architecture is referred to as encoder–decoder, with downsampling completed by an encoder and upsampling by a decoder. U-Net [63] is a variation of FCN that also shares the encoder–decoder architecture. The main difference between FCN and U-Net is how they complete skip connections. Skip connection combines feature maps in the encoder and decoder to aid the recovery of fine-grained details in the decoder. FCN completes skip connection by summing the corresponding feature maps, while U-Net concatenates them.

2) cGAN: Generative adversarial network (GAN) [64] consists of a generator to construct fake images and a discriminator to differentiate fake images from real ones. The generator and the discriminator are trained by competing with each other for a Nash equilibrium, i.e.,

$$\min_{G} \max_{D} V(G, D) \tag{8}$$

where G denotes the generator, D denotes the discriminator, and $V(\cdot, \cdot)$ denotes the loss function. GAN is first proposed as an unsupervised model, which transforms random noise to images. Then, cGAN [65] is proposed to guide the training with conditions. The generator of cGAN resembles the encoder–decoder architecture in Fig. 2 to take image inputs as conditions, and the training of the generator is guided by the discriminator to generate images close to those in the training set.

B. Congestion Prediction for Routability Optimization

In the task of congestion prediction, we further present a new metric, *peak NRMSE*, designed for routability-driven placement, and a new loss function, *biased loss*, to optimize this metric.

Studies in congestion prediction [8], [12], [15], [16] usually formulated the task as an image-to-image translation problem, since the congestion map can be viewed as an image-like array.

The commonly used metrics in these studies are the ones that assess image similarity, like NRMSE and SSIM which we have defined in Section II, from computer vision. However, in the practical application of the congestion prediction, the predicted congestion map is fed into the placer to guide cell inflation during routability-driven placement [16]. In this case, the highly congested region should be paid more attention to. Thus, it is questionable whether these metrics provide accurate assessment for prediction results.

1) Congestion Metric: In an effort to address this concern, we propose a new metric named "peak NRMSE." Inspired by the metric average congestion of g-cell edges (ACEs) from [66] and [67], which evaluates the top k percents average congestion in layout, we sort the elements in the congestion map by congestion value, then take out the top k ($k = 0.5, 1, 2, 5$) percents of the elements to calculate NRMSE, then average the results. In this way, we use only the most congested elements to compute NRMSE, promoting optimization in the localized congested regions

$$\text{peak NRMSE} = \frac{1}{4}\sum^{k}\frac{\sqrt{\frac{\sum^{topk}(y_{i,j}-\hat{y}_{i,j})^2}{h\times w}}}{y_{\max}-y_{\min}}. \tag{9}$$

2) Biased Loss for Congestion Learning: We design a loss to optimize peak NRMSE by paying more attention to the congested region in the congestion map, i.e., the element in the array with a high value.

The typical mean-squared error (MSE) loss can be formulated as

$$\text{MSE loss}(\hat{y}, y) = \frac{1}{n}\sum(\hat{y}_i - y_i)^2 \tag{10}$$

where y is the ground-truth value, $\hat{y}$ is the prediction of model, and i denotes the ith element in the array.

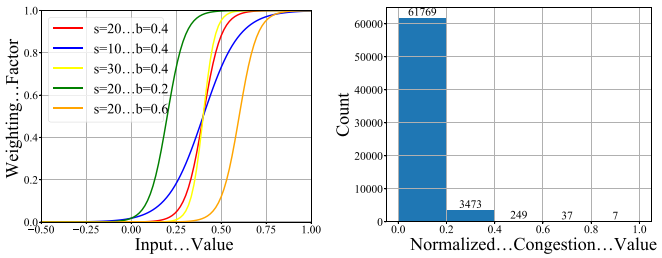
To improve optimization in highly congested regions, we develop a new loss function named "biased loss" by weighting the MSE loss with a sigmoid-like factor

$$\text{biased loss}(\hat{y}, y) = \frac{1}{n}\sum\frac{1}{1+\exp(s\cdot(b-y_i))}(\hat{y}_i - y_i)^2 \tag{11}$$

where s and b are hyperparameters to tune the shape of the weighting factor. s is the shrinkage factor that shrinks the horizontal coordinates of the original sigmoid function, and b is the bias factor that shifts the function with respect to vertical coordinates.

We show the weighting factor with different s and b in Fig. 8(a). When s is 10, the function changes smoothly. After raising s to 20, more rapid change is achieved, and when we further rise s, the function acts like a step function. Biased loss multiplies the elements with values greater than b in the ground truth congestion map by a weighting factor close to one during computing the loss and the others by a weighting factor close to zero. Noted that our congestion map is defined with overflow, so the elements with low congestion values still need attention. In other words, s, i.e., the rising speed of the weight, should be chosen carefully to reach a balance between local accuracy and global accuracy.

For all the congestion maps in CircuitNet, we first resize them to (256,256) and normalize them to [0,1], then plot the average histogram of all congestion maps, as shown in Fig. 8 (b). From the histogram, we can see about 95% of the values falls in [0,0.2], so b should at least be 0.4 to be "biased."



(a) (b)

Fig. 8. (a) Illustration of biased loss weighting factor regarding different input values with multiple sets of s and b. (b) Average normalized histogram of all the congestion maps in CircuitNet.

TABLE II
STATISTICS OF SUPERBLUE DESIGNS FROM THE ISPD 2015 CONTEST BENCHMARK

Designs	#Cells	#Nets	#Macros
superblue11_a	925,616	935,613	1,458
superblue12	506,097	511,606	286
superblue14	612,243	619,697	340
superblue16_a	680,450	697,303	419
superblue19	506,097	511,606	286

Furthermore, for the other tasks that also focus on the elements with high values, like the DRC violations prediction, and IR drop prediction, which are formulated as the hotspots prediction tasks with imbalanced positive and negative elements in training data, this loss can also be applied to achieve biased training and improve corresponding results.

C. Transfer Learning With Knowledge Distillation

The goal of transfer learning is to adapt models trained from one dataset to another dataset. This is a realistic demand in practice since it is not easy to obtain a large number of data samples for each design. In this work, we consider adapting the model trained from CircuitNet designs to ISPD 2015 contest designs, as the latter is widely adopted for routability-driven placement studies [2], [3], [16], [39]. The statistics of the superblue designs in the ISPD 2015 contest benchmark suite are shown in Table II, which are much larger than the RISC-V designs in CircuitNet.

We first verify the distributions of the two sets of designs in Fig. 9, where we plot the t-distributed stochastic neighbor embeddings (t-SNE) on the extracted congestion maps of these designs (all generated by Innovus, and the detailed settings are described in Section V-D). We can see that the distributions of the two datasets are partially overlapped. In other words, the two distributions are to some degree similar, but still different. Although the ISPD dataset contains much fewer data points, CircuitNet can only cover part of its distribution. To achieve high accuracy on the ISPD dataset, we need to adapt the knowledge learned from CircuitNet to ISPD, while using a minimum amount of data samples, due to the high cost of generating data samples.

TABLE III
RESULTS OF CONGESTION PREDICTION IN NRMSE, SSIM, AND PEAK NRMSE

Setting	NRMSE↓	SSIM↑	peak NRMSE↓				
			0.5%	1%	2%	5%	average
FCN with MSE loss [16]	**0.047**	**0.773**	0.441	0.323	0.236	0.155	0.289
cGAN with MSE loss [12]	0.055	0.739	0.420	0.314	0.237	0.165	0.284
FCN with biased loss ($s = 10$, $b = 0.4$)	0.059	0.735	0.337	0.244	0.179	**0.124**	0.221
FCN with biased loss ($s = 20$, $b = 0.4$)	0.105	0.616	0.267	**0.205**	**0.171**	0.157	**0.200**
FCN with biased loss ($s = 30$, $b = 0.4$)	0.183	0.421	**0.263**	0.229	0.222	0.237	0.238
FCN with biased loss ($s = 20$, $b = 0.2$)	0.057	0.740	0.367	0.264	0.190	0.126	0.237
FCN with biased loss ($s = 20$, $b = 0.6$)	0.238	0.330	0.311	0.320	0.335	0.350	0.329

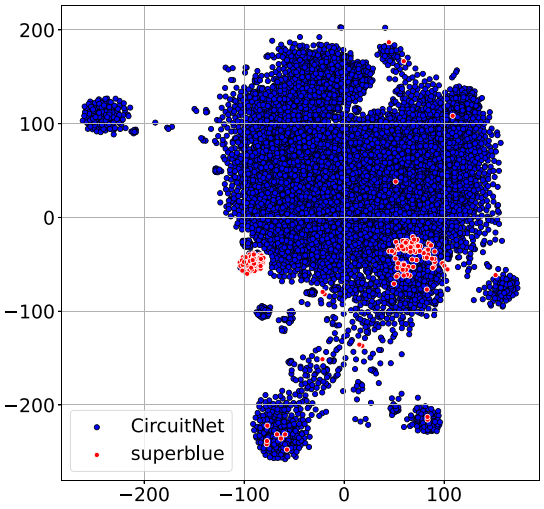


Fig. 9. t-SNE plot of congestion map for CircuitNet (blue) and `superblue` (red).

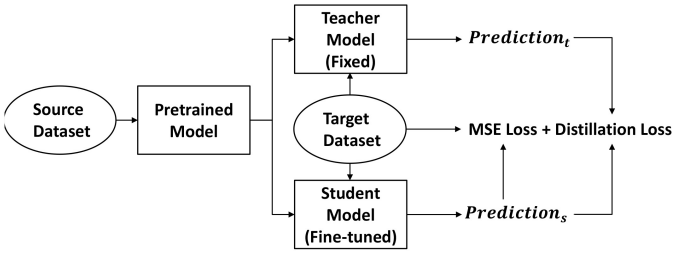


Fig. 10. Architecture of the proposed transfer learning framework.

To adapt ML models trained on a source dataset to a target dataset, we propose a transfer learning framework based on the teacher–student architecture for knowledge distillation. The architecture of the network is shown in Fig. 10. The model pretrained on the source dataset is duplicated into the teacher model and the student model, while the teacher model is fixed and the student model is further fine-tuned with the target dataset. In this way, the knowledge learned in the teacher network is transferred into the student network by aligning their outputs.

Besides the commonly used MSE loss in fine-tuning, a distillation loss, defined as the MSE between the output of teacher model $Precision_t$ and the output of student model $Precision_s$, is added to the optimization objective. We use the FCN-based model [16] as the backbones for the teacher and student networks. The teacher–student network is originally designed for training teacher and student networks on the same dataset [68], but our modification allows it to train the two networks on different datasets following similar distributions by regularizing the outputs of layers in the student network with the teacher network.

V. EXPERIMENTAL RESULTS

In this section, three parallel prediction tasks described in Section II are completed on CircuitNet. Results on models from previous work are compared based on metrics inherited from their work along with our proposed metrics *peak NRMSE*. Four designs, `RISCY-a`, `RISCY-b`, `RISCY-FPU-a`, and `RISCY-FPU-b`, are split into the train set, and the other two designs, `zero-riscy-a` and `zero-riscy-b` into the test set for all experiments. We train the models using PyTorch on a Linux machine with one NVIDIA A100 GPU. Transfer learning experiments on ISPD 2015 placement contest benchmarks are presented to verify the effectiveness of different transfer learning techniques, including the proposed transfer learning framework, in the congestion prediction task. We also verify the result of routability-driven placement on `superblue` designs from ISPD2015 with DREAMPlace [39] to provide an objective evaluation of the proposed metric, *peak NRMSE*, and the proposed loss function, *biased loss*.

A. Congestion Prediction

1) Experimental Setup: In this experiment, two congestion prediction models are compared.

1) The FCN-based model from [16] and [20].
2) The cGAN-based model from [12].

Three input features extracted at the placement stage: macro region, RUDY, and pin RUDY, are fed into models to predict congestion after global routing. In other words, the input features and labels are the same for all models in this experiment for a fair comparison. The side length of extracted feature maps ranges from about 200 to 300. To facilitate model training, in this and the following experiments, all feature maps and labels are resized to 256×256. The model from [12] is originally for routing congestion prediction in FPGA designs, using two features: 1) RUDY and 2) pin density. To fit our ASIC-based dataset, one new feature, macro region, is added to consider macro, and pin density is replaced with pin RUDY. In this way, its input is the same as the other two models. Moreover, its structure is modified slightly to fit the change in input and output channels.

2) Congestion Prediction Accuracy: Our experiment results are shown in Table III. Both two models show good performance on CircuitNet, which is consistent with their original study. The accuracy of cGAN in NRMSE and SSIM is relatively low, owing to the balance between the generator and the discriminator that prevent the generator from converging. But cGAN also shows competitive accuracy in peak NRMSE, which may be due to pixels with high values being paid more attention to by the discriminator.

We further explore the effect of biased loss and some other loss based on the FCN-based model from [16]. We train the model with biased loss with multiple groups of settings in s and b, and find out the best setting for optimizing peak NRMSE is $s = 20$ and $b = 0.4$. Though there is degradation in NRMSE and SSIM with the biased loss, the experimental results in Section V-E show that peak NRMSE is more consistent with the result of routability-driven placement, compared with NRMSE and SSIM.

B. DRC Violations Prediction

1) Experimental Setup: In this experiment, two DRC violation prediction models are compared.

1) The FCN-based model, RouteNet, from [20].
2) The U-Net-based model, J-Net, from [10].

For RouteNet, input features are macro region, RUDY, pin RUDY, cell density, and congestion, and the training label is DRC violations density in each tile. All of them are arrays of 256×256. Except for the aforementioned tile-based features, J-Net takes an additional feature, a high-resolution pin configuration map, which contains detailed locations and shapes of pins and pin blockages on the layout. To cape with larger layout dimensions in our dataset, the generated maps with a resolution of over $80\,000 \times 80\,000$ are cropped into smaller ones with a resolution of 6000×6000, which is consistent with the original setting in [10]. Following this configuration, the label for J-Net is also cropped into smaller ones with a size of 20×20.

2) DRC Prediction Accuracy: According to the problem formulation and performance metrics defined in Section II-F, we define that the tiles in labels with the number of DRC violations over a threshold 0.1 are labeled as hotspots, turning the label into a binary image. Note that the label is normalized into the range [0,1], then the threshold is 10% of the maximum DRC violation counts.

The results of the two models as binary classifiers are shown in Fig. 11 and Table IV. For J-Net, it fails to reach high performance in the ROC curve due to the fact that it takes cropped feature maps as input, so it cannot capture the effect of macros.

Then, we further train RouteNet with the biased loss, and the result is also shown in Fig. 11 and Table IV. 2% improvement in AUC can be reached with biased loss through paying more attention to the hotspot elements with high value, compared with the MSE loss.

C. IR Drop Prediction

1) Experimental Setup: In this experiment, one IR drop prediction model is tested.

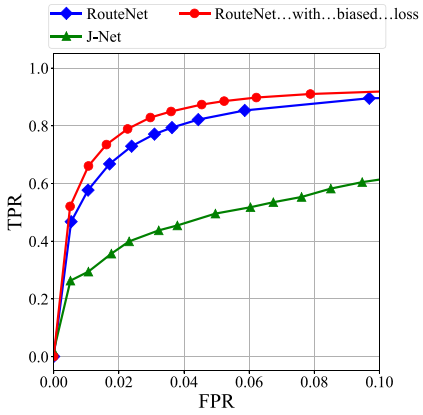


Fig. 11. ROC curves of models in DRC violations prediction.

TABLE IV
RESULTS OF DRC VIOLATION PREDICTION

Model	FPR (<=5%)	TPR (%)	Accuracy (%)	AUC (ROC)
RouteNet [20]	4.4	82.1	**95.8**	0.93
J-Net [10]	5.0	49.6	86.5	0.84
RouteNet with biased loss	4.5	**87.3**	**95.8**	**0.95**

1) The U-Net-based model, MAVIREC, from [32].

In MAVIREC, instance-level power report is cast into a 2-D array first, considering spatial information. Then, it is combined with signal arrival timing window and toggle rates to get a 3-D array. The third dimension represents the power distribution on each time interval derived from switching activities. Because IR drop is related to the switching activities of instances, larger local power demand may lead to more serious IR drop, enabling the prediction of IR drop.

In this experiment, the power report is provided by a vectorless dynamic analysis instead of a vector-based one. The 3-D power distribution array is used as the input feature for MAVIREC and our proposed model. And 2-D IR drop array is used as the label. The IR drop values are in logarithmic form to increase the difference between hotspots and nonhotspots to facilitate prediction.

2) IR Drop Prediction Accuracy: In practice, we want to constrain the IR drop under a certain percentage, usually 5% to 10% of the supply voltage, thus, we define that the tiles in the label with IR drop values over a threshold of 40 mV, which is 5% of the supply voltage, 810 mV, are labeled as hotspots, turning the label into a binary image. The results of MAVIREC are shown as ROC curve in Fig. 12 and Table V.

Then, we further train MAVIREC with the biased loss, and the result is also shown in Fig. 12 and Table V. 1% improvement in AUC can be reached with biased loss, compared with the MSE loss.

Besides MAVIREC, we also have experimented on PowerNet [29], which is a CNN-based model. However, PowerNet fails to train on our large-scale dataset, since its max-CNN structure requires unacceptable training time. CNN makes a prediction based on each pixel, so 256×256 iterations are necessary to train on one layout in our context. On the other hand, the result trained on a small portion of data has

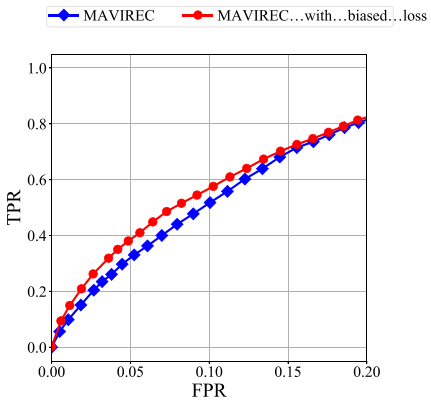


Fig. 12. ROC curve of the model in IR drop prediction.

TABLE V
RESULTS OF IR DROP PREDICTION

Model	FPR (<=5%)	TPR (%)	Accuracy (%)	AUC (ROC)
MAVIREC [32]	4.4	29.7	95.4	0.86
MAVIREC with biased loss	4.2	**35.0**	**95.7**	**0.87**

massive fluctuations, indicating that PowerNet lacks generalization ability. This result may be due to: 1) too small training set and 2) small windows used for training, causing the model to fail to consider the effect of macros. In contrast to CNN, the encoder–decoder-based structure can be trained on the whole layout in a single iteration, which is more efficient.

D. Transfer Learning

In this section, we evaluate the transfer learning techniques by transfer learning from CircuitNet to `superblue` designs from ISPD2015.

1) Comparison Setup: We compare train from scratch and five transfer learning methods, including:

1) train from scratch, denoted as "train";
2) pretraining and fine-tuning, denoted as "fine-tune";
3) pretraining and fine-tuning with fixed encoder layers, denoted as "freeze";
4) pretraining and fine-tuning with L2-SP-Fisher regularization, denoted as "EWC" [53];
5) pretraining and fine-tuning with L2-SP regularization, denoted as "L2";
6) pretraining and fine-tuning with the proposed teacher–student framework, denoted as "teacher–student."

L2-SP regularization and L2-SP-Fisher regularization are mentioned in Section II-E.

We took the `superblue` designs from the ISPD 2015 contest benchmark because they have over one million gates and have different distributions from the designs in CircuitNet. We generated layouts for each design and extracted the features through a similar procedure in Section III. For each design, we still set five core utilizations, i.e., {70%, 75%, 80%, 85%, 90%}, and six macro placements from the automatic floorplanning by Innovus. Eventually, 30 layouts can be obtained for each design.

TABLE VI
RESULTS OF TRANSFER LEARNING

Design	NRMSE↓					
	train	fine-tune	freeze	EWC	L2	teacher -student
superblue11_a	0.081	0.054	0.055	0.054	0.053	0.042
superblue14	0.089	0.058	0.060	0.061	0.055	0.049
superblue16_a	0.082	0.071	0.071	0.070	0.071	0.062
superblue19	0.092	0.058	0.058	0.058	0.054	0.046
Average	0.086	0.060	0.061	0.061	0.058	**0.050**
Design	SSIM↑					
	train	fine-tune	freeze	EWC	L2	teacher -student
superblue11_a	0.547	0.622	0.634	0.630	0.595	0.709
superblue14	0.382	0.517	0.509	0.495	0.533	0.534
superblue16_a	0.524	0.537	0.543	0.548	0.512	0.611
superblue19	0.449	0.573	0.577	0.564	0.572	0.621
Average	0.476	0.562	0.566	0.559	0.553	**0.619**
Design	peak NRMSE↓					
	train	fine-tune	freeze	EWC	L2	teacher -student
superblue11_a	0.113	0.105	0.105	0.104	0.109	0.099
superblue14	0.119	0.108	0.110	0.109	0.109	0.102
superblue16_a	0.257	0.287	0.285	0.281	0.294	0.289
superblue19	0.123	0.116	0.117	0.115	0.118	0.109
Average	0.153	0.154	0.154	0.152	0.158	**0.150**

We use the data from design `superblue12` for training/fine-tuning and the other four designs for testing. We first divide the 30 layouts of `superblue12` into six groups according to different macro placement solutions with each group containing five layouts. Then, for each group, we perform training/fine-tuning for 200 iterations. We report the average testing accuracy of models among all groups.

2) Experimental Results: The result is shown in Fig. 13 and Table VI. Compared with training from scratch, all the transfer learning methods can achieve much better accuracy in terms of NRMSE, SSIM, and peak NRMSE. The teacher–student network is able to distill the knowledge learned from CircuitNet to fit the distribution of `superblue` and outperforms the other fine-tuning methods by up to 20% in NRMSE, 10% in SSIM, and 5% in peak NRMSE.

E. Routability-Driven Placement

To further verify the effectiveness of the proposed techniques, we integrate the model obtained in Section V-D into an open-source placer, DREAMPlace [39], for routability optimization with cell inflation. We use our model to guide the cell inflation step by predicting the congestion map during global placement. Then, we feed the placement solutions into Cadence Innovus to finish the global routing and report the congestion rate (CR) and routed wirelength. The congestion map and the routed wirelength after global routing is reported by Innovus. CR is defined as

$$H-CR = \frac{\sum_{i=1}^{H}\sum_{j=1}^{V}\text{Overflow\_}h(i,j)}{\text{HV}} \tag{12a}$$

$$V-CR = \frac{\sum_{i=1}^{H}\sum_{j=1}^{V}\text{Overflow\_}v(i,j)}{\text{HV}} \tag{12b}$$

where H and V are horizontal and vertical Gcell counts. *Overflow_h* and *Overflow_v* are horizontal and vertical overflow values for Gcell (i, j).

Table VII summarizes the results of routability-driven placement. "DREAMPlace" denotes the results without ML

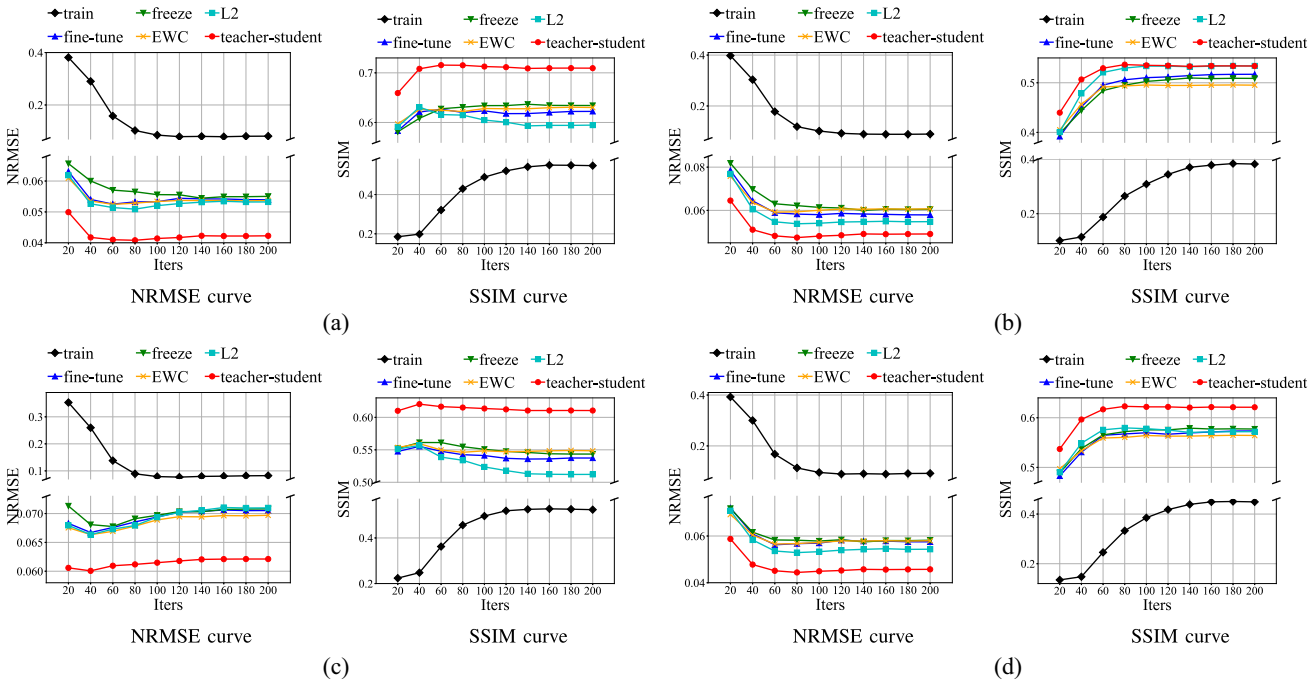


Fig. 13. NRMSE and SSIM curves for the methods used in the transfer learning of six groups of data, using the average results. (a) `superblue11_a`. (b) `superblue14`. (c) `superblue16_a`. (d) `superblue19`.

TABLE VII
RESULTS OF ROUTABILITY-DRIVEN PLACEMENT[2]

Benchmark	DREAMPlace [39]			DREAMPlace + FCN with MSE loss [16]			DREAMPlace + FCN with biased loss		
	H-CR	V-CR	WL (e+06 um)	H-CR	V-CR	WL (e+06 um)	H-CR	V-CR	WL (e+06 um)
`superblue11_a`	**0.0222**	0.0151	**38.27**	0.0239	0.0144	40.10	0.0225	**0.0135**	40.04
`superblue14`	0.1410	0.1508	**25.77**	0.1347	0.1481	25.87	**0.1325**	**0.1443**	25.83
`superblue16_a`	0.0719	0.0926	**28.48**	0.0595	0.0633	29.10	**0.0582**	**0.0618**	28.95
`superblue19`	0.1906	0.2638	**19.65**	0.1194	0.1811	20.16	**0.1174**	**0.1799**	20.07
Average	0.1064	0.1306	**28.04**	0.0844	0.1017	28.81	**0.0827**	**0.0999**	28.73
Ratio	1.287	1.307	**0.976**	1.021	1.018	1.003	**1.000**	**1.000**	1.000

[2] `superblue12` is used for training/fine-tuning, and the rest `superblue` designs are used to verify the placement results.

model routability optimization, while "FCN with MSE loss" and "FCN with biased loss" indicate the results of using ML models to guide cell inflation in routability optimization. The differences lie in the loss functions used to train the models. An average reduction of 2% in CR can be achieved with biased loss, compared with MSE loss. As the designs are highly congested, optimizing the congestion causes slight increase in routed wirelength. We ascribe the better congestion obtained from the model with biased loss to its optimization toward peak NRMSE rather than NRMSE, which is more consistent with the application scenarios of routability optimization in placement.

VI. CONCLUSION

In this article, we present CircuitNet, a large-scale open-source dataset for benchmarking ML applications in VLSI CAD, especially, for cross-stage prediction tasks in back-end design. With CircuitNet, we perform extensive experiments and comparisons among existing methods in routability prediction and IR drop prediction. We propose an evaluation metric to capture the top congested regions for routability-driven placement and a loss function for optimizing this metric during training. We further verify the transferability of features between different designs and propose a teacher–student framework to optimize the performance of transfer learning. In the future, we plan to enrich CircuitNet through long-term maintenance, such as adding new designs and data samples in advanced technology nodes and other ML tasks to increase the scale and diversity of the dataset.

REFERENCES

[1] X. He et al., "Ripple 2.0: High quality routability-driven placement via global router integration," in *Proc. ACM/EDAC/IEEE Design Autom. Conf. (DAC)*, 2013, pp. 1–6.

[2] C.-C. Huang et al., "NTUplace4dr: A detailed-routing-driven placer for mixed-size circuit designs with technology and region constraints," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst. (ICCAD)*, vol. 37, no. 3, pp. 669–681, Mar. 2018.

[3] C.-K. Cheng, A. B. Kahng, I. Kang, and L. Wang, "RePlAce: Advancing solution quality and routability validation in global placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst. (ICCAD)*, vol. 38, no. 9, pp. 1717–1730, Sep. 2019.

[4] Y. Hao et al., "Recent progress of integrated circuits and optoelectronic chips," *Sci. China Inf. Sci.*, vol. 64, no. 10, 2021, Art. no. 201401.

[5] A. B. Kahng, "Machine learning applications in physical design: Recent results and directions," in *Proc. Int. Symp. Phys. Design (ISPD)*, 2018, pp. 68–73.

[6] G. Huang et al., "Machine learning for electronic design automation: A survey," *ACM Trans. Design Autom. Electron. Syst.*, vol. 26, no. 5, pp. 1–46, 2021.

[7] M. Rapp et al., "MLCAD: A survey of research in machine learning for CAD keynote paper," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 10, pp. 3162–3181, Oct. 2022.

[8] C. Yu and Z. Zhang, "Painting on placement: Forecasting routing congestion using conditional generative adversarial nets," in *Proc. Annu. Design Autom. Conf. (DAC)*, 2019, pp. 1–6.

[9] Z. Zhou et al., "Congestion-aware global routing using deep convolutional generative adversarial networks," in *Proc. Workshop Mach. Learn. CAD (MLCAD)*, 2019, pp. 1–6.

[10] R. Liang et al., "DRC hotspot prediction at sub-10nm process nodes using Customized convolutional network," in *Proc. Int. Symp. Phys. Design (ISPD)*, 2020, pp. 135–142.

[11] C. Ma, Y. Xiao, S. Wang, J. Yu, and J. Chen, "CongestNN: An bi-directional congestion prediction framework for large-scale heterogeneous FPGAs," in *Proc. Int. Conf. ASIC*, 2021, pp. 1–4.

[12] M. B. Alawieh, W. Li, Y. Lin, L. Singhal, M. A. Iyer, and D. Z. Pan, "High-definition routing congestion prediction for large-scale FPGAs," in *Proc. Asia–South Pacific Design Autom. Conf. (ASP-DAC)*, 2020, pp. 26–31.

[13] C.-C. Chang et al., "Automatic routability predictor development using neural architecture search," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2021, pp. 1–9.

[14] C.-K. Cheng et al., "PROBE2.0: A systematic framework for routability assessment from technology to design in advanced nodes," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 5, pp. 1495–1508, May 2022.

[15] J. Chen, J. Kuang, G. Zhao, D. J.-H. Huang, and E. F. Young, "PROS: A plug-in for routability optimization applied in the state-of-the-art commercial EDA tool using deep learning," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2020, pp. 1–8.

[16] S. Liu, Q. Sun, P. Liao, Y. Lin, and B. Yu, "Global placement with deep learning-enabled explicit routability optimization," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2021, pp. 1821–1824.

[17] B. Wang et al., "LHNN: Lattice hypergraph neural network for VLSI congestion prediction," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2022, pp. 1297–1302.

[18] S. Yang et al., "Versatile multi-stage graph neural network for circuit representation," in *Proc. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 1–12.

[19] W.-T. J. Chan, P.-H. Ho, A. B. Kahng, and P. Saxena, "Routability optimization for industrial designs at sub-14nm process nodes using machine learning," in *Proc. ACM Int. Symp. Phys. Design (ISPD)*, 2017, pp. 15–21.

[20] Z. Xie et al., "RouteNet: Routability prediction for mixed-size designs using convolutional neural network," in *Proc. Int. Conf. Comput.-Aided Design (ICCAD)*, 2018, pp. 1–8.

[21] R. Islam and M. A. Shahjalal, "Predicting DRC violations using ensemble random forest algorithm," in *Proc. Annu. Design Autom. Conf. (DAC)*, 2019, pp. 1–2.

[22] T.-C. Yu et al., "Pin accessibility prediction and optimization with deep learning-based pin pattern recognition," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2019, pp. 1–6.

[23] A. F. Tabrizi, N. K. Darav, L. Rakai, I. Bustany, A. Kennings, and L. Behjat, "Eh?Predictor: A deep learning framework to identify detailed routing short violations from a placed netlist," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 6, pp. 1177–1190, Jun. 2020.

[24] W.-T. Hung, J.-Y. Huang, Y.-C. Chou, C.-H. Tsai, and M. Chao, "Transforming global routing report into DRC violation map with convolutional neural network," in *Proc. Int. Symp. Phys. Design (ISPD)*, 2020, pp. 57–64.

[25] K. Baek, H. Park, S. Kim, K. Choi, and T. Kim, "Pin accessibility and routing congestion aware DRC hotspot prediction using graph neural network and U-net," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2022, pp. 1–9.

[26] Z. Xie, R. Liang, X. Xu, J. Hu, Y. Duan, and Y. Chen, "Net2: A graph attention network method Customized for pre-placement net length estimation," in *Proc. Asia–South Pacific Design Autom. Conf. (ASP-DAC)*, 2021, pp. 671–677.

[27] Y.-C. Fang, H.-Y. Lin, M.-Y. Su, C.-M. Li, and E. J.-W. Fang, "Machine-learning-based dynamic IR drop prediction for ECO," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2018, pp. 1–7.

[28] C.-T. Ho and A. B. Kahng, "IncPIRD: Fast learning-based prediction of incremental IR drop," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2019, pp. 1–8.

[29] Z. Xie et al., "PowerNet: Transferable dynamic IR drop estimation via maximum convolutional neural network," in *Proc. Asia–South Pacific Design Autom. Conf. (ASP-DAC)*, 2020, pp. 13–18.

[30] X.-X. Huang, H.-C. Chen, S.-W. Wang, I. H.-R. Jiang, Y.-C. Chou, and C.-H. Tsai, "Dynamic IR-drop ECO optimization by cell movement with current waveform staggering and machine learning guidance," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2020, pp. 1–9.

[31] C.-H. Pao, A.-Y. Su, and Y.-M. Lee, "XGBIR: An XGBoost-based IR drop predictor for power delivery network," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2020, pp. 1307–1310.

[32] V. A. Chhabria, Y. Zhang, H. Ren, B. Keller, B. Khailany, and S. S. Sapatnekar, "MAVIREC: ML-aided vectored IR-drop estimation and classification," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2021, pp. 1825–1828.

[33] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.

[34] A. Krizhevsky, "Learning multiple layers of features from tiny images," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, Rep. TR-2009, 2009.

[35] C. Albrecht, "IWLS 2005 benchmarks," in *Proc. Int. Workshop Logic Synth. (IWLS)*, 2005, pp. 1–18.

[36] I. S. Bustany, D. Chinnery, J. R. Shinnerl, and V. Yutsis, "ISPD 2015 benchmarks with fence regions and routing blockages for detailed-routing-driven placement," in *Proc. Symp. Int. Symp. Phys. Design (ISPD)*, 2015, pp. 157–164.

[37] "ISPD 2016 routability-driven FPGA placement contest." 2016. [Online]. Available: http://www.ispd.cc/contests/16/ispd2016_contest.html

[38] Z. Chai, Y. Zhao, Y. Lin, W. Liu, R. Wang, and R. Huang, "CircuitNet: An open-source dataset for machine learning applications in electronic design automation (EDA)," *Sci. China Inf. Sci.*, vol. 65, no. 12, 2022, Art. no. 227401.

[39] Y. Lin et al., "DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 40, no. 4, pp. 748–761, Apr. 2021.

[40] P. Spindler and F. M. Johannes, "Fast and accurate routing demand estimation for efficient routability-driven placement," in *Proc. Design, Autom. Test Europe Conf. Exhibit. (DATE)*, 2007, pp. 1–6.

[41] C. Yang, G. Liu, C. Yan, and C. Jiang, "A clustering-based flexible weighting method in AdaBoost and its application to transaction fraud detection," *Sci. China Inf. Sci.*, vol. 64, no. 12, 2021, Art. no. 222101.

[42] X.-S. Wei, S.-L. Xu, H. Chen, L. Xiao, and Y. Peng, "Prototype-based classifier learning for long-tailed visual recognition," *Sci. China Inf. Sci.*, vol. 65, no. 6, 2022, Art. no. 160105.

[43] H. Yang, L. Luo, J. Su, C. Lin, and B. Yu, "Imbalance aware lithography hotspot detection: A deep learning approach," *J. Micro/Nanolith. MEMS MOEMS*, vol. 16, no. 3, p. 1, 2017.

[44] B. Wang, L. Jiang, W. Zhu, L. Guo, J. Chen, and Y.-W. Chang, "Two-stage neural network classifier for the data imbalance problem with application to hotspot detection," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2021, pp. 175–180.

[45] H. Yang, J. Su, Y. Zou, Y. Ma, B. Yu, and E. F. Y. Young, "Layout hotspot detection with feature tensor generation and deep biased learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 6, pp. 1175–1187, Jun. 2019.

[46] H. Chen and D. Boning, "Online and incremental machine learning approaches for IC yield improvement," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2017, pp. 786–793.

[47] R. Liang, H. Xiang, J. Jung, J. Hu, and G.-J. Nam, "A stochastic approach to handle non-determinism in deep learning-based design rule violation predictions," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2022, pp. 1–8.

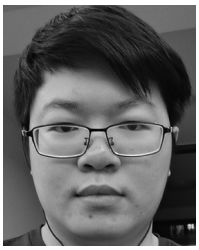
[48] D. Hyun, Y. Jung, I. Cho, and Y. Shin, "Decoupling capacitor insertion minimizing IR-drop violations and routing DRVs," in *Proc. Asia–South Pacific Design Autom. Conf. (ASP-DAC)*, 2023, pp. 271–276.

[49] Z. Jiang, M. Liu, Z. Guo, S. Zhang, Y. Lin, and D. Pan, "A tale of EDA's long tail: Long-tailed distribution learning for electronic design automation," in *Proc. Workshop Mach. Learn. CAD (MLCAD)*, 2022, pp. 135–141.

[50] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

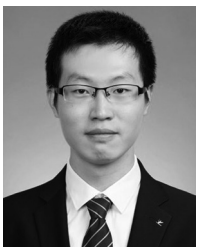
[51] F. Zhuang et al., "A comprehensive survey on transfer learning," *Proc. IEEE*, vol. 109, no. 1, pp. 43–76, Jan. 2021.

[52] K. He, R. Girshick, and P. Dollar, "Rethinking ImageNet pre-training," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 4917–4926.
[53] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, 2017.
[54] X. Li et al., "DELTA: DEep learning transfer using feature map with attention for convolutional networks," 2020, *arXiv:1901.09229*.
[55] Y. Lin et al., "Data efficient lithography modeling with transfer learning and active data selection," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst. (ICCAD)*, vol. 38, no. 10, pp. 1900–1913, Oct. 2019.
[56] T. Gai et al., "Flexible hotspot detection based on fully convolutional network with transfer learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst. (ICCAD)*, vol. 41, no. 11, pp. 4626–4638, Nov. 2022.
[57] J. Liu, M. Hassanpourghadi, Q. Zhang, S. Su, and M. S.-W. Chen, "Transfer learning with Bayesian optimization-aided sampling for efficient AMS circuit modeling," in *Proc. Int. Conf. Comput.-Aided Design (ICCAD)*, 2020, pp. 1–9.
[58] H. Geng, Q. Xu, T.-Y. Ho, and B. Yu, "PPATuner: Pareto-driven tool parameter auto-tuning in physical design via Gaussian process transfer learning," in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2022, pp. 1237–1242.
[59] Y. Lin, T. Qu, Z. Lu, Y. Su, and Y. Wei, "Asynchronous reinforcement learning framework and knowledge transfer for net-order exploration in detailed routing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 9, pp. 3132–3142, Sep. 2022.
[60] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, pp. 600–612, 2004.
[61] A. Traber et al., *PULPino: A Small Single-Core RISC-V SoC*, ETH Zürich, Zürich, Switzerland, 2016.
[62] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 4, pp. 640–651, Apr. 2017.
[63] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," 2015, *arXiv:1505.04597*.
[64] I. J. Goodfellow et al., "Generative adversarial networks," 2014, *arXiv:1406.2661*.
[65] M. Mirza and S. Osindero, "Conditional generative adversarial nets," 2014, *arXiv:1411.1784*.
[66] Y. Wei et al., "GLARE: Global and local wiring aware routability evaluation," in *Proc. Annu. Design Autom. Conf. (DAC)*, 2012, pp. 768–773.
[67] N. Viswanathan, C. Alpert, C. Sze, Z. Li, and Y. Wei, "The DAC 2012 routability-driven placement contest and benchmark suite," in *Proc. Annu. Design Autom. Conf. (DAC)*, 2012, p. 774.
[68] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.



Zhuomin Chai received the B.S. degree in physics from Wuhan University, Wuhan, China, in 2020, where he is currently pursuing the Ph.D. degree with the School of Physics and Technology.

He is a visiting student with Peking University, Beijing, China. His research interests include physical designs and machine learning in CAD.



Yuxiang Zhao received the B.E. degree in optoelectronic information science and engineering from the China University of Petroleum, Beijing, China, in 2019, and the M.E. degree in control engineering from the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen, China, in 2022. He is currently pursuing the Ph.D. degree with the School of Integrated Circuits, Peking University, Beijing.

His research interests involve deep learning and applications, and physical design algorithms and implementations.

Wei Liu received the B.S., M.S., and Ph.D. degrees from the School of Physics and Technology, Wuhan University, Wuhan, China, in 2000, 2003, and 2008, respectively.

He is currently serving as a Professor and the Department Head of the Department of Microelectronics, School of Physics and Technology, Wuhan University. His current research interests include digital–analog hybrid chip design, sensor chip design, novel computing architecture for AI acceleration, and EDA.

Yibo Lin (Member, IEEE) received the B.S. degree in microelectronics from Shanghai Jiao Tong University, Shanghai, China, in 2013, and the Ph.D. degree in electrical and computer engineering from the University of Texas at Austin, Austin, TX, USA, in 2018.

He is currently an Assistant Professor with the School of Integrated Circuits, Peking University, Beijing, China. His research interests include physical design, machine learning applications, and heterogeneous computing in VLSI CAD.

Dr. Lin is a recipient of the Best Paper Awards at premier EDA/CAD journals/conferences, such as IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, DAC, DATE, and ISPD.

Runsheng Wang (Member, IEEE) received the B.S. and Ph.D. (Highest Hons.) degrees from Peking University, Beijing, China, in 2005 and 2010, respectively.

From November 2008 to August 2009, he was a Visiting Scholar with Purdue University, West Lafayette, IN, USA. He joined Peking University in 2010, where he is currently a Full Professor with the School of Integrated Circuits, where he also serves as the Associate Dean of the School of EECS. He has authored/coauthored one book, four book chapters, and over 190 scientific papers, including more than 40 papers published in International Electron Devices Meeting (IEDM) and Symposium on VLSI Technology (VLSI-T). He has been granted over 20 U.S. patents and over 30 Chinese patents. His current research interests include nanoscale CMOS devices, characterization and reliability, design-technology co-optimization and EDA, emerging technologies, and circuits for new-paradigm computing.

Dr. Wang was awarded the IEEE EDS Early Career Award by the IEEE Electron Device Society (EDS), the National Distinguished Young Scholars by the National Natural Science Foundation of China, the Natural Science Award (First Prize) by the Ministry of Education (MOE) of China, and many other awards. He serves on the editorial board of IEEE TRANSACTIONS ON ELECTRON DEVICES and *Science China Information Sciences*. He has also served on the Technical Program Committee of many IEEE conferences, including IEDM, IRPS, EDTM, and IPFA.

Ru Huang (Fellow, IEEE) received the B.S. (Hons.) and M.S. degrees in electronic engineering from Southeast University, Nanjing, China, in 1991 and 1994, respectively, and the Ph.D. degree in microelectronics from Peking University, Beijing, China, in 1997.

Since 1997, she has been a Faculty Member with Peking University, where she is currently a Professor. She has also been serving as the President of Southeast University since 2022. She has authored or coauthored five books, five book chapters, and more than 300 articles, including more than 80 articles in IEDM (35 IEDM articles from 2007 to 2019), VLSI Technology Symposium, IEEE ELECTRON DEVICE LETTERS, and IEEE TRANSACTIONS ON ELECTRON DEVICES. She has delivered over 50 keynote/invited talks at conferences and seminars. She holds over 200 patents, including 49 U.S. patents. Her research interests include nano-scaled CMOS devices, ultralow-power new devices, new devices for neuromorphic computing, emerging memory technology, and device variability/reliability.

Dr. Huang is an Elected Academician of the Chinese Academy of Sciences and an Elected Member of TWAS Fellow.